Submission 2: A reusable three-gene visualization function

Chunlei Li

Purpose and scope

The same exposure and groups are retained. The three available genes are CYP24A1 (vitamin D metabolism), PTX3 (inflammatory response), and CXCR2 (chemokine signaling). They were chosen for biological context and availability, not by ranking the observed p-values. This submission contains nine plots.

Import and data preparation

```
# All cleaning starts from the three unchanged instructor CSV files.
library(ggplot2)
library(dplyr)
library(tidyr)
library(patchwork)
dir.create("results", showWarnings = FALSE)
dir.create("figures", showWarnings = FALSE)

cov_path <- "data/raw/GSE85307_covar-1.csv"
if (!file.exists(cov_path)) cov_path <- "data/raw/GSE85307_covar.csv"
covar_raw <- read.csv(cov_path, stringsAsFactors = FALSE, check.names = FALSE)
expr_raw <- read.csv("data/raw/GSE85307_geneExpression.csv",
                     row.names = 1, check.names = FALSE)
link_raw <- read.csv("data/raw/GSE85307_geneNames.csv",
                     stringsAsFactors = FALSE, check.names = FALSE)

# Remove the label before the FIRST colon, then trim surrounding whitespace.
strip_label <- function(x) trimws(sub("^[^:]*:\\s*", "", x, perl = TRUE))
clean <- covar_raw
for (v in setdiff(names(clean), c("subjectID", "geoID"))) {
  clean[[v]] <- strip_label(clean[[v]])
}
num_vars <- c("mat_age_yrs", "delivery_gestAge_wks",
              "sampleCollection_gestAge_wks", "mat_BMI", "mat_vitD")
for (v in num_vars) {
  before <- clean[[v]]
  clean[[v]] <- suppressWarnings(as.numeric(before))
  if (any(is.na(clean[[v]]) & !is.na(before))) stop("Numeric conversion failed: ",
   ↪  v)
}
stopifnot(!anyDuplicated(clean$geoID), !anyDuplicated(clean$subjectID),
          !anyDuplicated(colnames(expr_raw)), !anyDuplicated(rownames(expr_raw)),
          !anyDuplicated(link_raw$PROBEID), !anyDuplicated(link_raw$GENE),
```

```
          setequal(clean$geoID, colnames(expr_raw)),
          all(clean$preg_preeclampsia %in% c("Normal", "Preeclampsia")),
          all(clean$mat_asthma %in% c("0", "1")))
expr <- as.matrix(expr_raw)
stopifnot(is.numeric(expr), all(is.finite(expr)))

# Match expression columns by ID: never assume the input order is the same.
expr <- expr[, match(clean$geoID, colnames(expr)), drop = FALSE]
stopifnot(identical(colnames(expr), clean$geoID))
map <- link_raw[match(rownames(expr), as.character(link_raw$PROBEID)), ]
stopifnot(!anyNA(map$GENE), identical(as.character(map$PROBEID), rownames(expr)))
rownames(expr) <- map$GENE

# Study-specific threshold from Mirzakhani et al. (2016), NOT a diagnostic rule.
clean <- clean %>% mutate(
  preg_preeclampsia = factor(preg_preeclampsia, levels = c("Normal",
   ↪  "Preeclampsia")),
  vitD_group = factor(ifelse(mat_vitD < 30, "<30 ng/mL", ">=30 ng/mL"),
                      levels = c("<30 ng/mL", ">=30 ng/mL")),
  mat_asthma = factor(mat_asthma, levels = c("0", "1"), labels = c("No", "Yes")),
  batch_Microarray = factor(batch_Microarray))
# Preserve all 157 samples; do not remove outliers just to improve the plot.
all_genes <- as.data.frame(t(expr), check.names = FALSE)
analysis_data <- bind_cols(clean, all_genes)
selected_genes <- c("CYP24A1", "PTX3", "CXCR2")
heat_genes <- c(selected_genes, "JAK2", "LYN", "SOCS5", "IL7", "IL2", "TGM2",
 ↪  "TIMP1")
stopifnot(all(heat_genes %in% rownames(expr)), nrow(analysis_data) == 157L)
missing_table <- data.frame(variable = names(clean), missing =
 ↪  colSums(is.na(clean)))
write.csv(missing_table, "results/missingness.csv", row.names = FALSE)
write.csv(map, "results/probe_gene_mapping_used.csv", row.names = FALSE)
write.csv(clean, "results/clean_covariates.csv", row.names = FALSE)
write.csv(as.data.frame(table(clean$preg_preeclampsia, clean$vitD_group)),
          "results/group_counts.csv", row.names = FALSE)
write.csv(data.frame(file = c(cov_path, "data/raw/GSE85307_geneExpression.csv",
                              "data/raw/GSE85307_geneNames.csv"),
                     md5 = unname(tools::md5sum(c(cov_path,
                     "data/raw/GSE85307_geneExpression.csv",
                      ↪  "data/raw/GSE85307_geneNames.csv")))),
          "results/input_checksums.csv", row.names = FALSE)
```

Reusable function

Inputs are a data frame, one or more gene names, one continuous column name, and exactly two categorical column names. The output is a named list of ggplots.

```
# Return a named list of three ggplot objects for each requested gene.
plot_gene_panels <- function(data, genes, continuous, categorical) {
  genes <- unlist(genes, use.names = FALSE)
  if (!is.data.frame(data) || !is.character(genes) || length(genes) < 1L ||
      length(continuous) != 1L || length(categorical) != 2L) {
    stop("Supply a data frame, >=1 gene, one continuous name, and two categorical
     ↪  names.")
```

```r
  }
  if (anyDuplicated(genes)) stop("Gene names must be unique.")
  if (!all(c(genes, continuous, categorical) %in% names(data))) stop("Unknown
   ↪  column name.")
  if (!is.numeric(data[[continuous]]) || !all(vapply(data[genes], is.numeric,
   ↪  logical(1)))) {
    stop("Gene expression and the continuous covariate must be numeric.")
  }
  if (!all(vapply(data[categorical], function(x) is.factor(x) || is.character(x),
   ↪  logical(1)))) {
    stop("Categorical covariates must be factors or character columns.")
  }
  out <- list()
  for (gene in genes) {
    # Filter only the columns needed for this gene and state the plotted n.
    d <- data[complete.cases(data[, c(gene, continuous, categorical)]), , drop =
↪   FALSE]
    if (nrow(d) < 3L) stop("Too few complete observations for ", gene)
    xlab <- if (continuous == "mat_vitD") "Baseline 25(OH)D (ng/mL)" else
↪   continuous
    ylab <- paste0(gene, " expression (provided scale)")
    n_txt <- paste("n =", nrow(d))
    style <- theme_classic(base_size = 11, base_family = "sans") +
      theme(plot.title = element_text(face = "bold", size = 13),
            plot.subtitle = element_text(size = 10, colour = "grey35"),
            legend.position = "top", legend.title = element_blank(),
            strip.background = element_rect(fill = "grey94", colour = NA),
            strip.text = element_text(face = "bold"),
            plot.margin = margin(10, 14, 10, 10))
    hist <- ggplot(d, aes(x = .data[[gene]])) +
      geom_histogram(bins = 24, fill = "#337C8E", colour = "white", linewidth =
       ↪  0.25) +
      labs(title = paste(gene, "expression distribution"), subtitle = n_txt,
           x = ylab, y = "Number of participants") + style
    scatter <- ggplot(d, aes(x = .data[[continuous]], y = .data[[gene]])) +
      geom_point(colour = "#337C8E", alpha = 0.65, size = 1.7) +
      geom_smooth(method = "lm", formula = y ~ x, se = TRUE,
                  colour = "#B65E27", fill = "#E4B48B", linewidth = 0.8) +
      labs(title = paste(gene, "and", if (continuous == "mat_vitD") "baseline
       ↪  vitamin D" else continuous), subtitle = n_txt,
           x = xlab, y = ylab) + style
    # Use both categories: x = outcome; facet and colour = vitamin D group.
    counts <- d %>% group_by(across(all_of(categorical))) %>% summarise(n = n(),
↪   .groups = "drop")
    ymax <- max(d[[gene]])
    span <- diff(range(d[[gene]])); if (span == 0) span <- 1
    counts$label_y <- ymax + 0.08 * span
    box <- ggplot(d, aes(x = .data[[categorical[1]]], y = .data[[gene]],
                         fill = .data[[categorical[2]]])) +
      geom_boxplot(width = 0.55, outlier.shape = NA, alpha = 0.65, linewidth =
       ↪  0.45) +
      geom_point(position = position_jitter(width = 0.12, height = 0, seed = 103),
                 shape = 21, size = 1.3, alpha = 0.6, stroke = 0.2) +
      geom_text(data = counts, aes(x = .data[[categorical[1]]], y = label_y,
                                   label = paste0("n = ", n)),
```

```
                inherit.aes = FALSE, size = 3.1) +
      facet_wrap(vars(!!rlang::sym(categorical[2]))) +
      scale_fill_manual(values = rep(c("#337C8E", "#D5A13B", "#7D5596", "#AD4E32",
       ↪  "#57A773", "#C27D9F"),
        length.out = length(unique(d[[categorical[2]]])))) +
      scale_y_continuous(expand = expansion(mult = c(0.07, 0.17))) +
      labs(title = paste(gene, "by outcome and vitamin D group"), subtitle = n_txt,
           x = if (categorical[1] == "preg_preeclampsia") "Pregnancy outcome" else
            ↪  categorical[1], y = ylab) + style + theme(legend.position = "none")
    out[[gene]] <- list(histogram = hist, scatterplot = scatter, boxplot = box)
  }
  out
}

# Save vector PDF and 300-dpi PNG versions for manuscripts and presentations.
save_gene_panels <- function(panels) {
  for (gene in names(panels)) for (kind in names(panels[[gene]])) {
    base <- file.path("figures", paste(gene, kind, sep = "_"))
    w <- if (kind == "boxplot") 8.2 else 7
    ggsave(paste0(base, ".pdf"), panels[[gene]][[kind]], width = w, height = 4.7)
    ggsave(paste0(base, ".png"), panels[[gene]][[kind]], width = w, height = 4.7,
     ↪  dpi = 300)
  }
}
```

```
# A character vector or a list of names is accepted.
panels <- plot_gene_panels(analysis_data, list("CYP24A1", "PTX3", "CXCR2"),
                           "mat_vitD", c("preg_preeclampsia", "vitD_group"))
stopifnot(length(panels) == 3L, sum(lengths(panels)) == 9L)
save_gene_panels(panels)
```

CYP24A1

```
panels[["CYP24A1"]][["histogram"]]
```

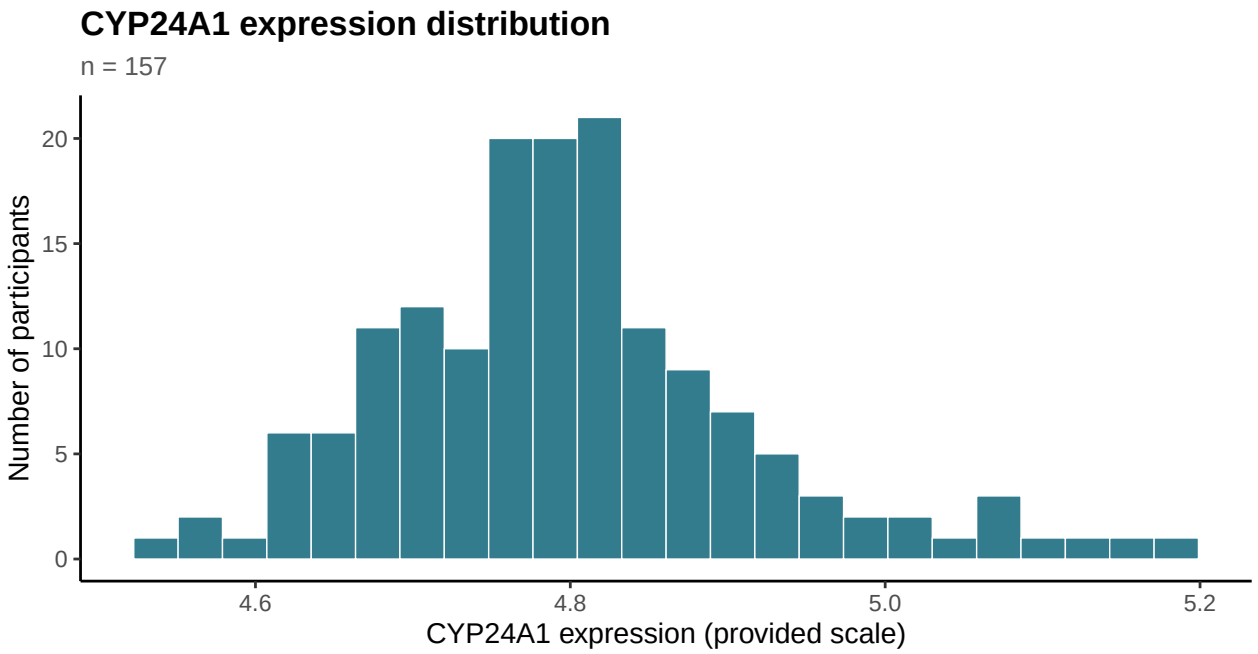


Figure 1: CYP24A1: Expression distribution; 24 histogram bins.

```
panels[["CYP24A1"]][["scatterplot"]]
```

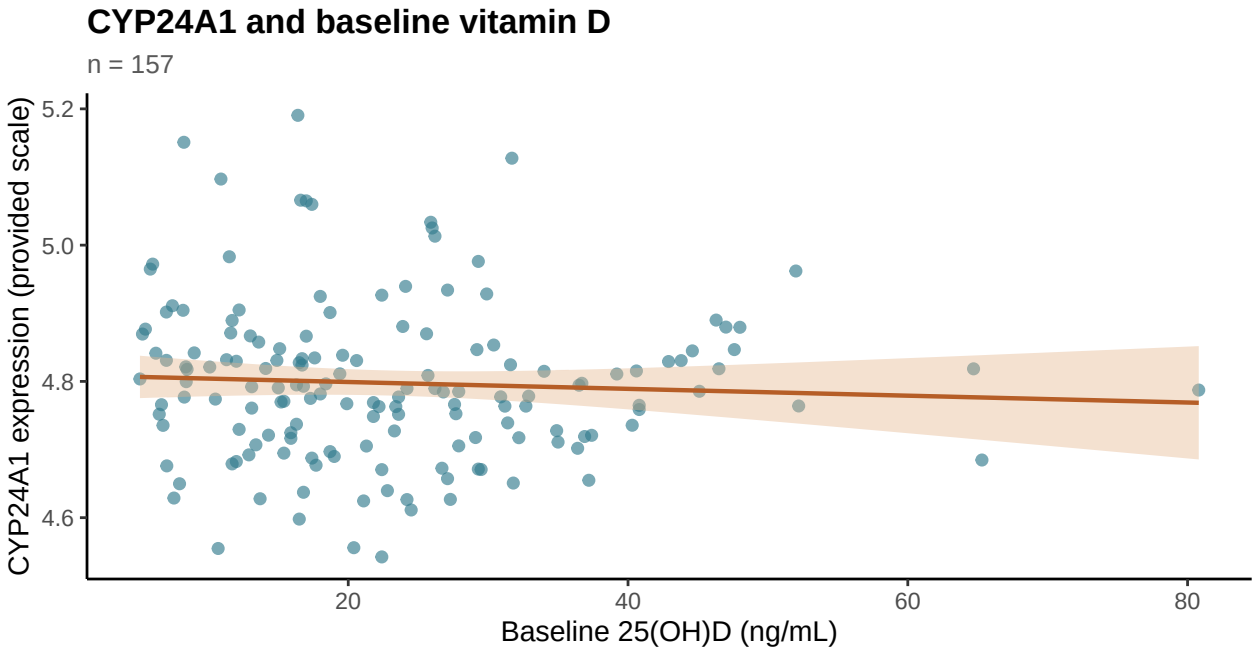


Figure 2: CYP24A1: Baseline 25(OH)D versus expression; unadjusted linear fit and model-based 95% confidence band.

```
panels["CYP24A1"]["boxplot"]
```

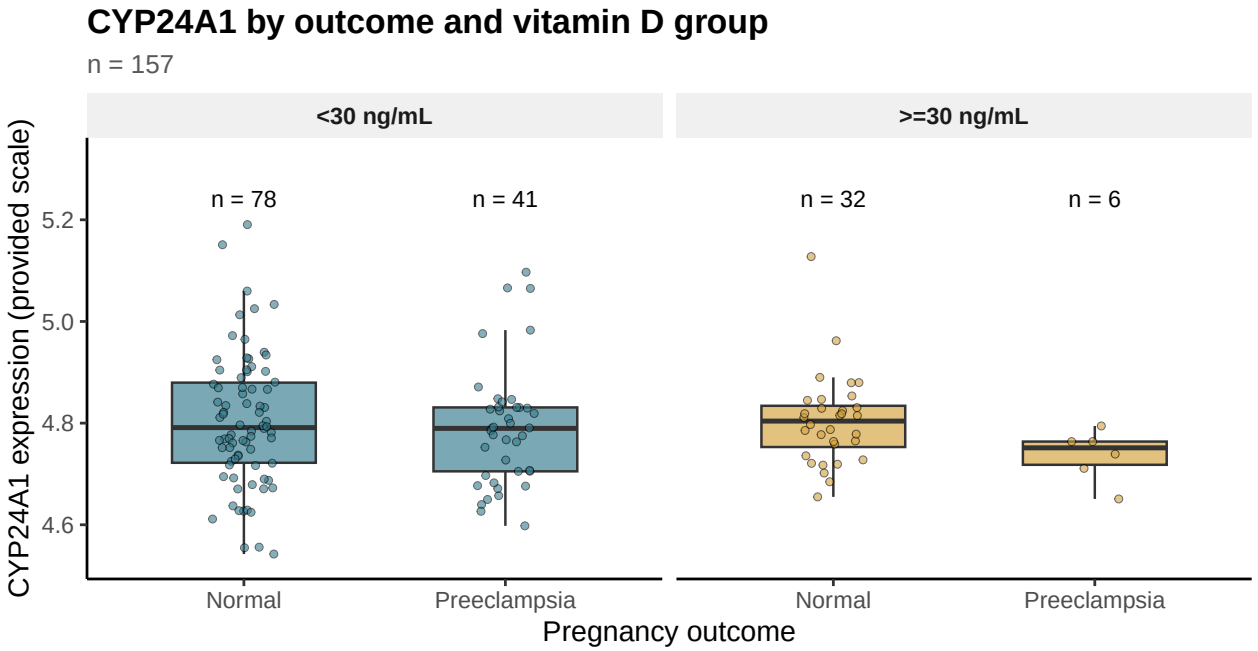


Figure 3: CYP24A1: Expression by both categorical covariates; all observations and group counts are displayed.

PTX3

```
panels[["PTX3"]][["histogram"]]
```

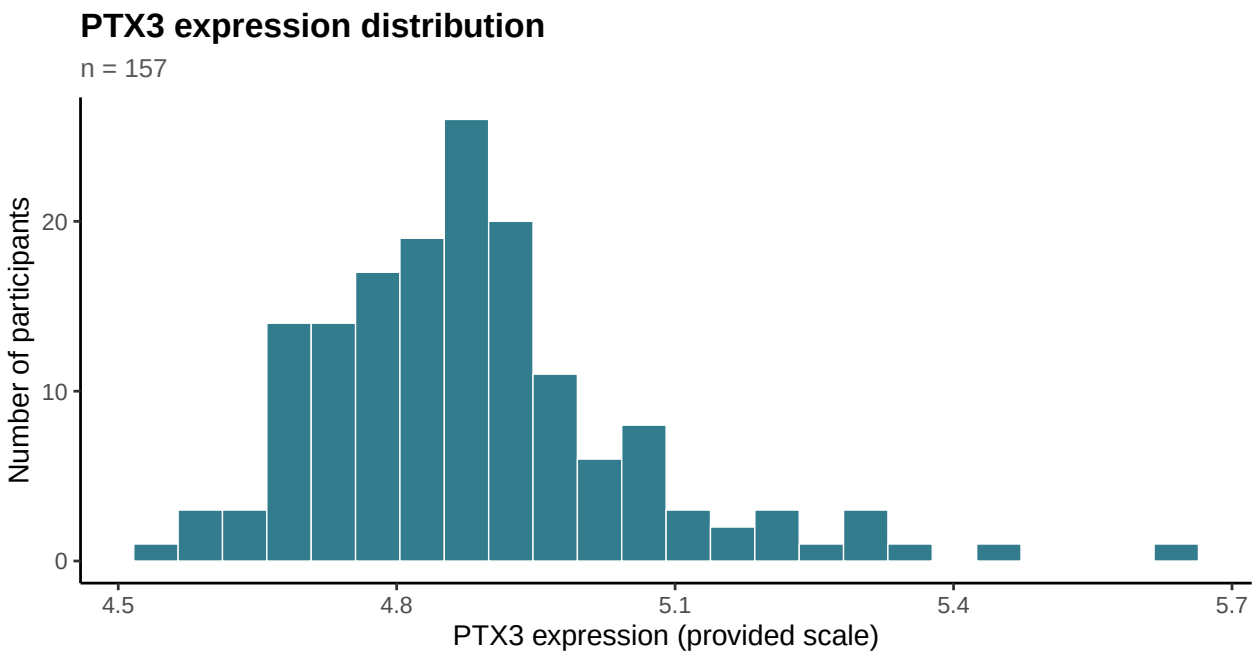


Figure 4: PTX3: Expression distribution; 24 histogram bins.

```
panels[["PTX3"]][["scatterplot"]]
```

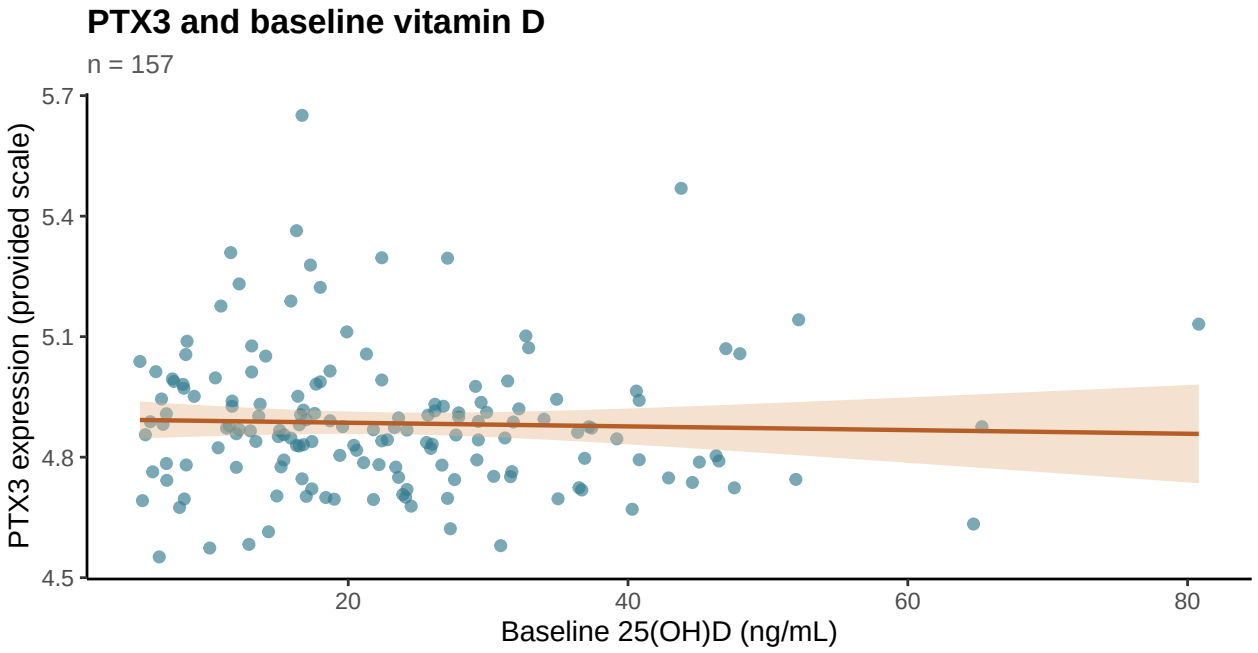


Figure 5: PTX3: Baseline 25(OH)D versus expression; unadjusted linear fit and model-based 95% confidence band.

```
panels["PTX3"]["boxplot"]
```

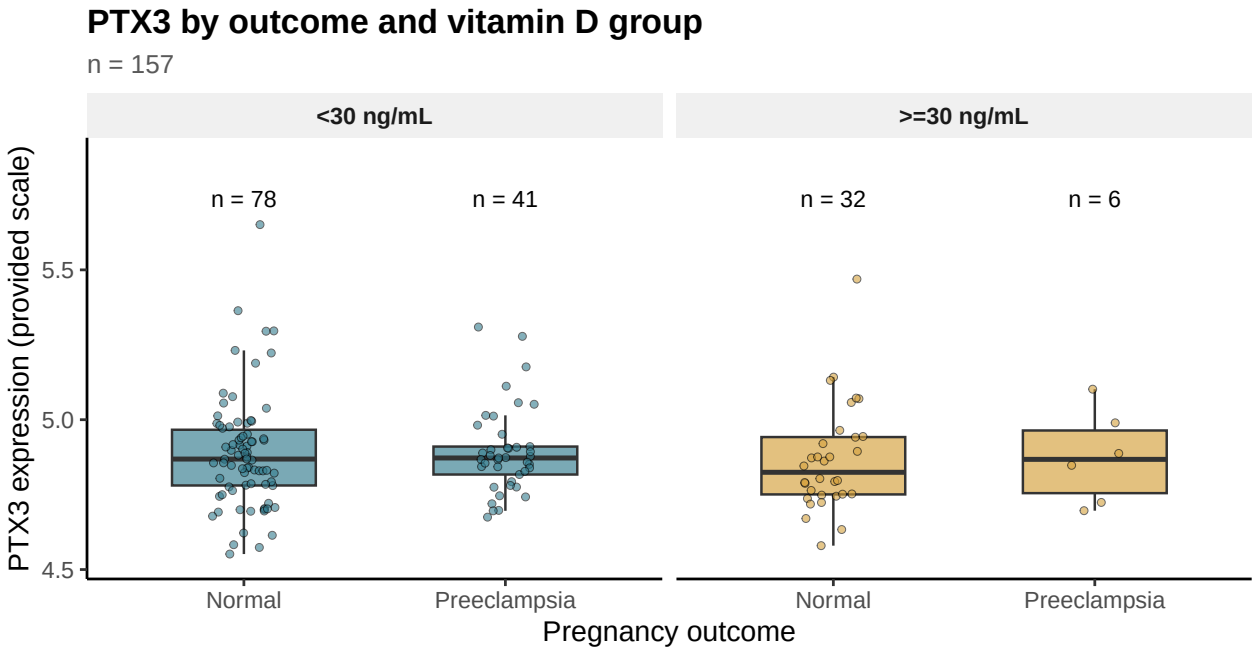


Figure 6: PTX3: Expression by both categorical covariates; all observations and group counts are displayed.

CXCR2

```
panels[["CXCR2"]][["histogram"]]
```

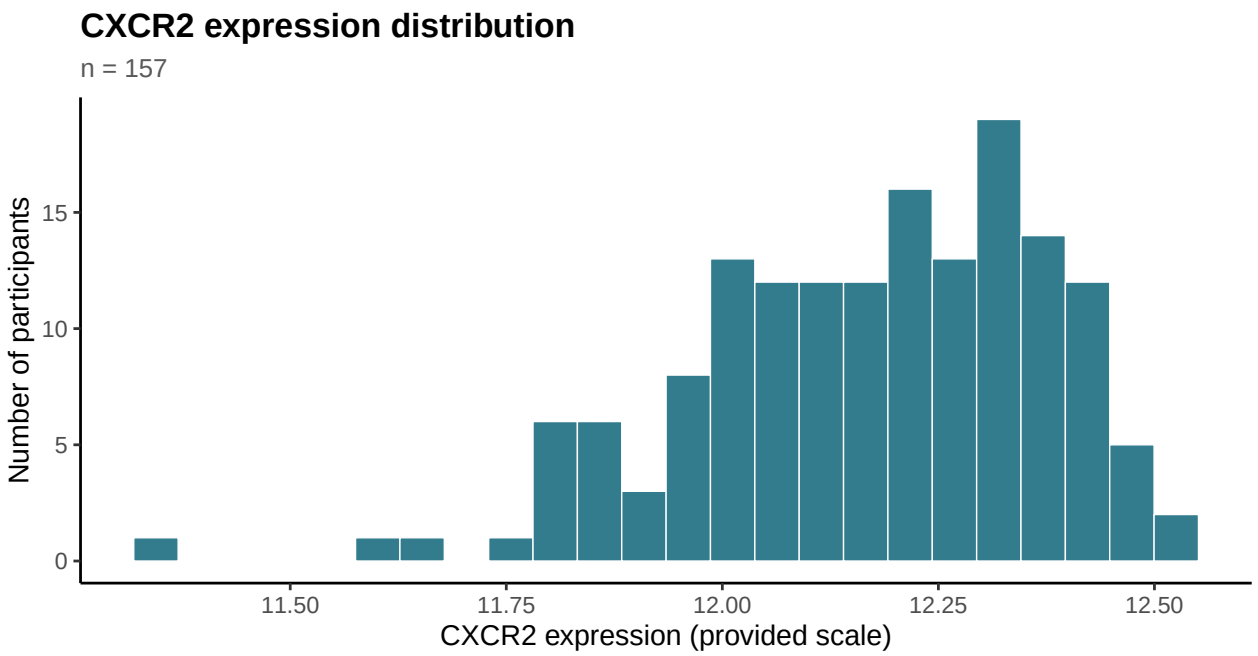


Figure 7: CXCR2: Expression distribution; 24 histogram bins.

```
panels[["CXCR2"]][["scatterplot"]]
```

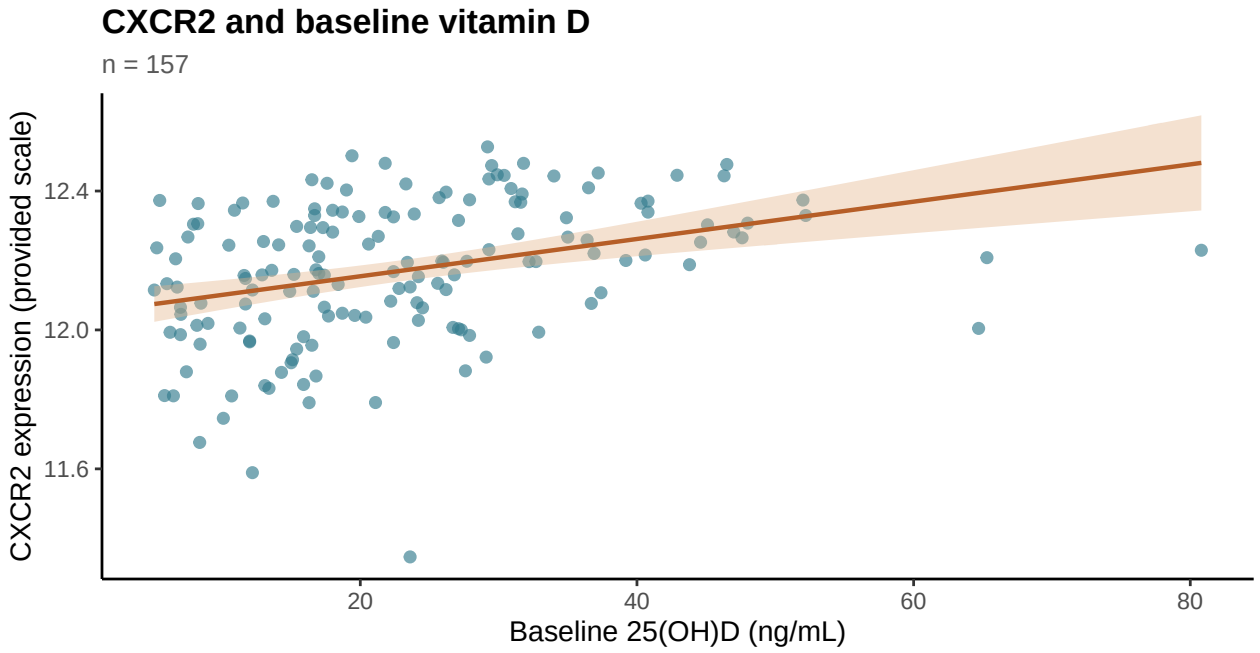


Figure 8: CXCR2: Baseline 25(OH)D versus expression; unadjusted linear fit and model-based 95% confidence band.

```
panels["CXCR2"]["boxplot"]
```

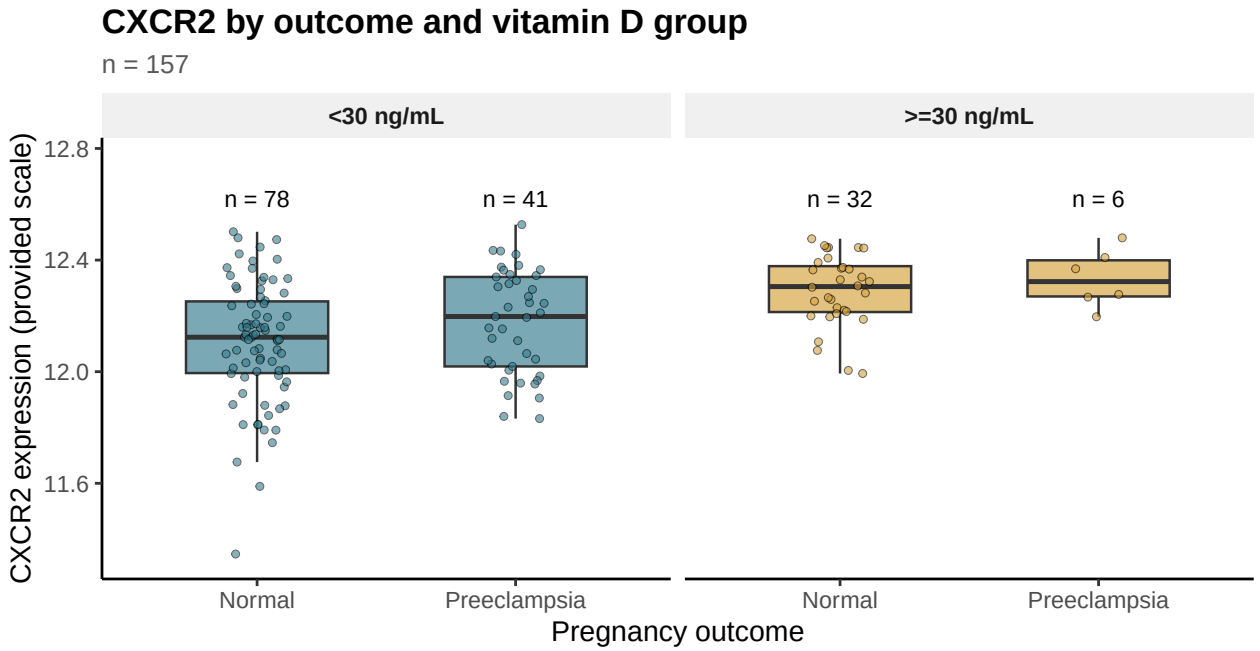


Figure 9: CXCR2: Expression by both categorical covariates; all observations and group counts are displayed.

Interpretation and feedback record

CYP24A1 and PTX3 have little apparent linear trend with vitamin D in these plots. CXCR2 shows a positive descriptive trend. The groups remain overlapping, and the high-vitamin-D preeclampsia subgroup has only six observations. No adjustment for confounding, batch or multiple testing is claimed.

The initial version uses consistent typography, all-point overlays, sample counts, and PDF/300-dpi exports. These are design decisions, not a claim that instructor feedback has been incorporated. Record actual Submission 1 feedback and the resulting revisions in `FEEDBACK_LOG.md` before submitting this stage.

Submission notes

Prepared with ChatGPT/Codex assistance; review and disclose assistance according to course policy. Original study: Mirzakhani et al. (2016), doi:10.1172/JCI89031. Gene context: NCBI Gene IDs 1591, 5806 and 3579. Software details follow.

```
sessionInfo()
```

```
#> R version 4.3.3 (2024-02-29)
#> Platform: x86_64-pc-linux-gnu (64-bit)
#> Running under: Ubuntu 24.04.3 LTS
#>
#> Matrix products: default
#> BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
#> LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
#>
#> locale:
#>  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
#>  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
#>  [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
#> [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
#>
#> time zone: America/Los_Angeles
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] patchwork_1.2.0 tidyr_1.3.1     dplyr_1.1.4     ggplot2_3.4.4
#>
#> loaded via a namespace (and not attached):
#>  [1] Matrix_1.6-5     gtable_0.3.4     compiler_4.3.3   tinytex_0.49
#>  [5] tidyselect_1.2.0 splines_4.3.3    scales_1.3.0     yaml_2.3.8
#>  [9] fastmap_1.1.1    lattice_0.22-5   R6_2.5.1         labeling_0.4.3
#> [13] generics_0.1.3   knitr_1.45       tibble_3.2.1     munsell_0.5.0
#> [17] pillar_1.9.0     rlang_1.1.3      utf8_1.2.4       xfun_0.41
#> [21] cli_3.6.2        withr_2.5.0      magrittr_2.0.3   mgcv_1.9-1
#> [25] digest_0.6.34    grid_4.3.3       lifecycle_1.0.4  nlme_3.1-164
#> [29] vctrs_0.6.5      evaluate_0.23    glue_1.7.0       farver_2.1.1
#> [33] fansi_1.0.5      colorspace_2.1-0 rmarkdown_2.25   purrr_1.0.2
#> [37] tools_4.3.3      pkgconfig_2.0.3  htmltools_0.5.7
```